

# AI 504 – Proof trees for $\mathcal{A}$ and graphs

Stefano Volpe

2026-03-31

We work with  $\mathcal{A}$ , the language of “All  $p$  are  $q$ ”, using Haskell. We have four goals:

1. modelling proof trees;
2. modelling graphs;
3. implementing the all-graph of a set of formulas;
4. implementing the reflexive-transitive closure of a graph.

**Grading.** The submitted code will be inspected and tested. A grade of 2 or higher means that you have submitted a compiling program declaring all identifiers you are asked to implement. On top of which, a 4 means that you have understood all questions. A 7 means a correct answer to at least one third of the (sub)problems. A 10 means a correct answer to at least two thirds of the (sub)problems. A twelve means a correct answer to all problems. Grade 0 is awarded on an ad-hoc basis. The total problem set grade will be an average of the programming grade and the written homework grade.

**Instructions.** All instructions from the parent problem set apply. Any way of receiving “intelligent” help via a computer is expressly forbidden. The following are the only allowed `import` statements: you can use any declaration imported via these.

```
1 import Data.Foldable
2 import Data.List.NonEmpty as NE
3 import Data.Set as S
```

The first module, `Foldable`, is dedicated to foldable structures. Useful operations, such as `find`, can be performed on these. The second module, `NonEmpty`, is dedicated to non-empty lists. As a path is a particular kind of non-empty list of vertices, non-empty lists will come in handy. We used the `as` keyword to nickname some of the imported modules, so that we can qualify ambiguous occurrences of imported identifiers with shorter module names.

Please paste a (corrected) version of your past solution below these import statements. You will need part of it. If you want to rerun its tests, you should prepend the `fromList` invocation with `S.`, so as to avoid ambiguities. You are always allowed to invoke previously defined functions (from the current homework, or the previous one).

## Problem 1: The type of proof trees

Define a unary type constructor `ProofTree` following Definition 1.10. If  $p$  is a type, then `ProofTree p` must be a type describing all possible proof trees of logic  $\mathcal{A}$  using  $p$  as a signature. In particular:

- if  $a :: p$  and  $b :: p$ , then `Assume a b :: ProofTree p` represents an assumption leaf;
- if  $a :: p$ , then `Axiom a :: ProofTree p` represents an application of `AXIOM` deriving “All  $a$  are  $a$ ”;
- if  $d1 :: ProofTree p$  and  $d2 :: ProofTree p$ , then `Barbara d1 d2` represents an application of `BARBARA` whose left and right subtree are  $d1$  and  $d2$  respectively.

Only expressions constructed using either of the three rules above should have type `ProofTree p`.

*Example 1.1:* Let `Char` be the set of all Unicode code points. Then  $\mathcal{D}$ , defined as

$$\frac{\frac{\text{All 'a' are 'b'}}{\text{All 'a' are 'b'}} \text{AXIOM} \quad \text{All 'b' are 'b'}}{\text{All 'a' are 'b'}} \text{BARBARA} \quad \frac{\text{All 'b' are 'c'}}{\text{All 'b' are 'c'}} \text{BARBARA}}{\text{All 'a' are 'c'}} \text{BARBARA}$$
 is a proof tree with signature `Char`.

```

1 d :: ProofTree Char
2 d = Barbara
3     (Barbara
4       (Assume 'a' 'b')
5       (Axiom 'b'))
6       (Assume 'b' 'c')

```

Here are two conditions that proof trees must satisfy, but that do not need to be enforced by your type constructor definition:

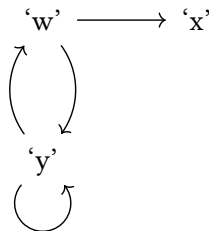
- all premises should be elements of some fixed set  $\Gamma$ ;
- in a barbara application, the second noun in the root of the left subtree should coincide with the first noun in the root of the right subtree.

In a proper implementation, we would use *encapsulation* to ensure the user cannot violate either of these constraints.

## Problem 2: The type (alias) of graphs

Recall that declarations starting with keyword `type` introduce an alias for a pre-existing type (constructor), rather than generating a new one. Define a unary type constructor alias `Graph`. If  $v$  is a type, then `Graph v` represents the type of all graphs whose set of vertices is given by all expressions of type  $v$ , according to Definition 1.17. Hence, `Graph v` must be an alias for the type of all finite<sup>1</sup> sets of ordered pairs whose two components both have type  $v$ . Each pair is meant to represent an edge in the graph, and we use ordered pairs because all our graphs are directed. Note that we don't need to memorise the set of all vertices in a term of type `Graph v`, as that's already given by  $v$ .

*Example 2.1:* It holds that  $G$ , defined as



is a graph with `Char` as a set of vertices.  
(Vertices with no outgoing or incoming edges are not depicted.)

```

1 g :: Graph Char
2 g = S.fromList [
3     ('w', 'x'),
4     ('w', 'y'),
5     ('y', 'w'),
6     ('y', 'y')
7 ]

```

## Problem 3: The all-graph of a set of formulas

Please add the following two deriving declarations to your program.

```

1 deriving instance (Eq p) => Eq (Sentence p)
2 deriving instance (Ord p) => Ord (Sentence p)

```

Whenever a certain type  $p$  implements typeclass `Eq`, the first line ensures Haskell derives the “reasonable” implementation of `Eq` for `Sentence p`, as well. This is *component-wise equality*, i.e. `AllAre a b == AllAre c d` if and only if `a == c` and `b == d`. So `AllAre 1 5 == AllAre 1 5` is `True`, but `AllAre 1 2 == AllAre 1 3` is `False`.

<sup>1</sup>Much like in the previous homework, we work with modules that only support finite sets. However, graphs in mathematics might have an infinite number of edges.

Whenever a certain type  $p$  implements typeclass `Ord`, the second line ensures Haskell derives a “reasonable” implementation of `Ord` for `Sentence p`, as well. This is *lexicographic comparison*, i.e. to compare `AllAre a b` to `AllAre c d`, we compare `a` to `c` first, and then only compare `b` to `d` if we need to break a tie because `a == c`. So `AllAre 1 5 < AllAre 2 3` is `True`, and so is `AllAre 1 5 < AllAre 1 7`.

Implement a function `allGraph` parametric in a type variable  $p$  representing a signature. This function should implement Definition 1.9. Assuming  $p$  implements `Ord`, the function should accept as an input a set of sentences over signature  $p$ . Then, `allGraph` should return the all-graph of the input graph whose vertices have type  $p$ .

*Example 3.1:* The all-graph of

$$\{\text{All 'w' are 'x'}, \text{All 'w' are 'y'}, \\ \text{All 'y' are 'w'}, \text{All 'y' are 'y'}\}$$

is  $G$  from Example 2.1.

```
1 -- >>> allGraph (S.fromList [AllAre 'w' 'x', AllAre 'w' 'y', AllAre 'y' 'w', AllAre 'y' 'y']) == g
2 -- True
```

## Problem 4: Reflexive-transitive closure

In a few weeks, we will implement a decider for derivability in  $\mathcal{A}$ ’s proof system (Algorithm 1.32). Such an algorithm involves the notion of *reflexive-transitive closure* (Definition 1.20), so that’s what we would like to focus on in this problem. Because Algorithm 1.32’s definition is somewhat vague, it is not immediately clear that some of its operations involving such closure cannot be expressed merely in terms of a decider<sup>2</sup> for reflexive-transitive closure. All closure-related operations in the algorithm can instead be expressed in terms of a function computing the *set of paths with a fixed ending vertex*. This is the goal of this problem.

In the following, we will use a generic type  $v$  to represent vertices. When implementing functions involving vertices, you should use  $v$  or some other type variable name to be parametric in some type representing vertices.

We will always encode edges as ordered pairs of vertices, and paths as non-empty lists of vertices. Use the unary type constructor `NonEmpty` for non-empty lists. These can be constructed using the infix constructor `(:|) :: v -> [v] -> NonEmpty v`, where the left operand represents the head of the list, and the right operand represents the tail. (Remember not to use those parentheses when applying `:|` to two arguments. They are only needed when one or both arguments are missing, like in the signature just stated.)

### 4.1 Deciding whether a path can be prepended with an edge

Assume  $v$  implements `Eq` (all types implementing `Ord` must implement `Eq`, as well). Define an uncurried<sup>3</sup> decider for the “...can be prepended with...” binary relation. The first argument must be

<sup>2</sup>Recall that a decider for a decision problem (i.e., a yes/no question) is a function/program that, on every input, terminates and outputs the correct yes/no answer. In our case, a decider would check whether a pair of vertices belongs to the reflexive-transitive closure of the graph.

<sup>3</sup>A function with  $n$  arguments is said to be *curried* when its type is of the form `a_1 -> ... -> a_n -> b = a_1 -> (... -> (a_n -> b))`. When  $n \geq 1$ , these functions can hence be seen as unary functions which, when applied to a single argument, return curried functions in  $n - 1$  arguments. In contrast, an *uncurried* function of  $n$  arguments has type `(a_1, ..., a_n) -> b`, so it is implemented as a unary function which takes a tuple as an input. Our mathematical examples will always use uncurried functions, as those are the most common in everyday mathematics, even when the corresponding Haskell snippets will be using curried functions.

an edge in a graph whose vertices are expressions of type  $v$ . (Recall that edges are ordered pairs of vertices.) The second argument must be a path in said graph. The function, named `isPrependable`, outputs `True` if it is possible to place the edge right in front of the path to obtain a new, longer path, and `False` otherwise.

*Example 4.1.1:* Let `Char` be the set of vertices. Then,

`isPrependable (('a', 'b'), 'b' → 'i' → 'd') = True`  
`isPrependable (('a', 'l'), 'b' → 'i' → 'd') = False`

```
1 -- >>> isPrependable (('a', 'b'), 'b' :| "id")
2 -- True
3 -- >>> isPrependable (('a', 'l'), 'b' :| "id")
4 -- False
```

## 4.2 Prepending a path with an edge

Do not make any assumptions on  $v$ . Define an uncurried binary function `prepend` which prepends a path with an edge (i.e., places the edge before the path), assuming this is possible. The first argument must be an edge. The second must be a path. Since we are assuming that this operation is possible, it does not matter what your function does when the operation is impossible.

*Example 4.2.1:* Let  $\mathbb{Z}$  be the set of vertices.

`prepend((0, 1), 1 → 2 → 4) = 0 → 1 → 2 → 4`

```
1 -- >>> prepend ((0,1), 1 :| [2,4])
2 -- 0 :| [1,2,4]
```

## 4.3 Compute all prepend candidates

Assume  $v$  implements `Eq`. Define a curried binary function `prependables` that, given a set of edges and a set of paths as first and second argument respectively, computes the binary relation “...can be prepended with...” as a set of pairs whose first component is an edge from the first argument, and whose second component is a path from the second argument.

*Example 4.3.1:* Let  $\mathbb{Z}$  be the set of vertices.

`prependables ({(0, 1)(2, 2)}, {1 → 3 → 5, 5 → 4 → 5, 1 → 4, 2 → 9}) = {`  
 `((0, 1), 1 → 3 → 5),`  
 `((0, 1), 1 → 4),`  
 `((2, 2), 2 → 9)}`

```
1 -- >>> prependables (S.fromList [(0,1), (2,2)]) (S.fromList [1:|[3,5], 5:|[4,5], 1:|[4], 2:|[9]])
2 -- fromList [((0,1),1 :| [3,5]),((0,1),1 :| [4]),((2,2),2 :| [9])]
```

## 4.4 Adding prepending results to a set of paths

Assume  $v$  implements `Ord`. Define a curried binary function `addPrependResults` whose two arguments are a graph  $g$  and a set  $s$  of paths in  $g$ . The function should return the set of all paths in  $g$  that satisfy at least one of the following two conditions:

1. they belong to  $s$ ;
2. they can be obtained by correctly prepending some element of  $s$  with an edge in  $g$ .

*Example 4.4.1:* Consider  $G$  from Example 2.1. It holds that

$$\begin{aligned} \text{addPrependResults } (G, \{ 'y' \rightarrow 'w' \rightarrow 'x' \}) = \{ \\ & 'w' \rightarrow 'y' \rightarrow 'w' \rightarrow 'x', \quad \langle \text{condition 2} \rangle \\ & 'y' \rightarrow 'w' \rightarrow 'x', \quad \langle \text{condition 1} \rangle \\ & 'y' \rightarrow 'y' \rightarrow 'w' \rightarrow 'x' \} \quad \langle \text{condition 2} \rangle \end{aligned}$$

```
1 -- >>> addPrependResults g (S.fromList ['y':|"wx"])
2 -- fromList ['w' :| "ywx", 'y' :| "wx", 'y' :| "ywx"]
```

## 4.5 Repeated application

Define a ternary curried function `iterateN` parametric in an arbitrary type `a` (which does not need to represent a type of vertices). The arguments to this function must be, in this order:

- a number `n` of type `Int` which is assumed to be non-negative;
- a unary function `f` from `a` to `a`;
- a value `x :: a`.

If the assumptions are met, the function must return the result of applying `f` to `x` for a total of `n` times in a row. As per usual, if they are not met (i.e., `n` is negative), your function can behave as you like.

*Example 4.5.1:* If  $f : \mathbb{Z} \rightarrow \mathbb{Z}$  is defined as  $f(x) = 2x$ , then

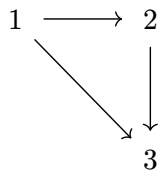
$$\text{iterateN}(3, f, 1) = 2 * (2 * (2 * 1)) = 8.$$

```
1 -- >>> iterateN 3 (2 *) 1
2 -- 8
```

## 4.6 Set of paths with a fixed ending vertex

Assume `v` implements `Ord`. Define a curried binary function `pathsTo` that, given a graph `g` as a first argument, and a vertex `b` as a second argument, returns the set of all paths in `g` that end in `b`. If there are multiple paths ending in `b` and sharing the same starting vertex, it is enough include at least one of said paths in the output set. (You, the implementer, have therefore some freedom when choosing the output.)

*Example 4.6.1:* At least one of the possible correct definitions for `pathsTo` is such that, if graph  $H$  is defined as



then

$$\text{pathsTo}(H, 3) = \{1 \rightarrow 2 \rightarrow 3, 1 \rightarrow 3, 2 \rightarrow 3, 3\}.$$

```
1 -- >>> pathsTo (S.fromList [(1,2),(1,3),(2,3)]) 3
2 -- fromList [1 :| [2,3], 1 :| [3], 2 :| [3], 3 :| []]
```

Note that a definition for `nonTrivialPaths` may not output both of the two distinct paths from 1 to 3, and still be correct. It is enough to output only one of the two.